

TASK 2: API Integration & JSON Handling

Goal:

Learn how Python communicates with external APIs and handles JSON data.

Objective:

- Learn API integration using Python.
- Learn JSON Handling
- Fetch data from a web service using the Requests library.
- Parse and process JSON responses.
- Implement user-based search functionality.
- Handle network and API errors effectively.

Requirements

- Fetch data using Requests library
- Parse JSON responses
- Apply filtering or search logic
- Handle API errors

Technologies Used

- Python
- Requests Library
- JSON
- OpenWeather API

Project Files:

API-Integration-Project/

```
|  
|— weather_api.py  
|— README.md
```

GitHub Repository:

<https://github.com/bejugamsameera-lgtm/API-Integration-JSON-Handling.git>

LinkedIN Post:

https://www.linkedin.com/posts/bejugam-sameera-2b8350363_alfidotech-pythondeveloper-python-ugcPost-7462409949336485889-Y2gJ?utm_source=share&utm_medium=member_desktop&rcm=ACoAAFPJKqwBKmowDO0w9vo76hdH8xQvuoi3e3c

Step-by-Step Process

Step 1: Create Project Folder

Create a new folder named:

API-Integration-Project

Step 2: Install Python

Check whether Python is installed:

`python --version`

Step 3: Install Requests Library

Open Terminal/Command Prompt and run:

`pip install requests`

if needed:

`python -m pip install requests`

Step 4: Get an API Key

1. Visit [OpenWeather](#)
2. Create a free account.
3. Generate an API key.
4. Copy the API key.

5. Step 5: Create Python File

6. Create a file named:
7. `weather_api.py`

Paste the API integration code into this file.

CODE SNIPPET

```
import requests
```

```
API_KEY = " YOUR_API_KEY " # Replace with your  
OpenWeather API key
```

```
CITY = input("Enter city name: ")
```

```
url =  
f'https://api.openweathermap.org/data/2.5/weather?q={CITY}&appid  
={API_KEY}&units=metric'
```

```
try:
```

```
    response = requests.get(url, timeout=10)
```

```
    # Check if request was successful
```

```
    if response.status_code == 200:
```

```
        data = response.json()
```

```
        # Parse JSON data
```

```
        city = data["name"]
```

```
        temperature = data["main"]["temp"]
```

```
        humidity = data["main"]["humidity"]
```

```
        weather = data["weather"][0]["description"]
```

```
        # Display output
```

```
print("\nWeather Information")
print("-----")
print(f'City: {city}')
print(f'Temperature: {temperature}°C')
print(f'Humidity: {humidity}%')
print(f'Condition: {weather}')
```

```
elif response.status_code == 404:
```

```
    print("City not found. Please enter a valid city name.")
```

```
else:
```

```
    print(f'Error: {response.status_code}')
```

```
except requests.exceptions.RequestException as e:
```

```
    print("Network error:", e)
```

Step 6: Add API Key

Replace:

```
API_KEY = "YOUR_API_KEY"
```

with your actual API key:

Step 7: Save the File

Save the Python file in the project folder.

Step 8: Run the Program

Open Terminal in the project folder and execute:

```
python weather_api.py
```

Step 9: Enter City Name

Example:

Enter city name: Hyderabad

The program sends a request to the API server.

Step 10: Receive JSON Response

The API returns weather data in JSON format.

Example:

```
{
  "name": "Hyderabad",
  "main": {
    "temp": 31.4,
    "humidity": 62
  },
  "weather": [
    {
      "description": "scattered clouds"
    }
  ]
}
```

Step 11: Parse JSON Data

Convert JSON into a Python dictionary:

```
data = response.json()
```

Extract required information:

```
city = data["name"]
```

```
temperature = data["main"]["temp"]
```

```
humidity = data["main"]["humidity"]
```

```
weather = data["weather"][0]["description"]
```

Step 12: Display Output

Example output:

Weather Information

City: Hyderabad

Temperature: 31.4°C

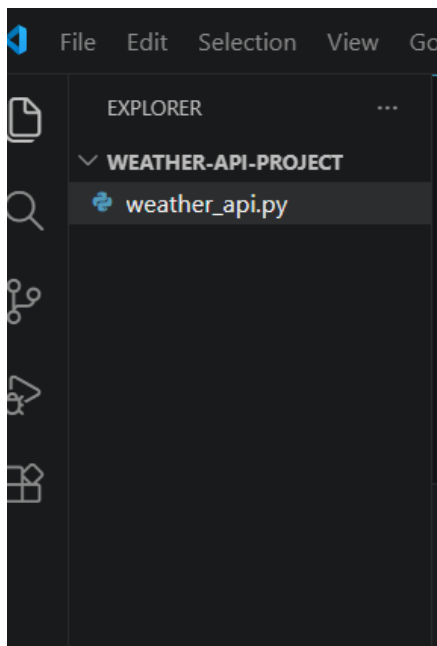
Humidity: 62%

Condition: scattered clouds

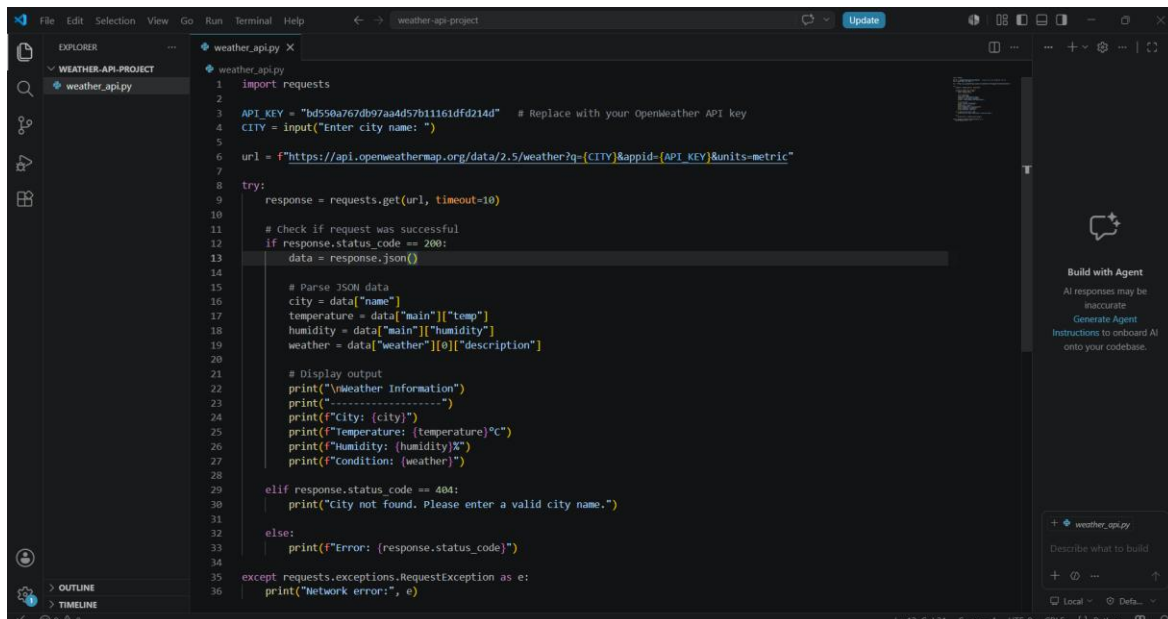
SCREENSHOTS

Screenshot 1: Folder Structure

Project folder containing the Python script used for API Integration, json handling

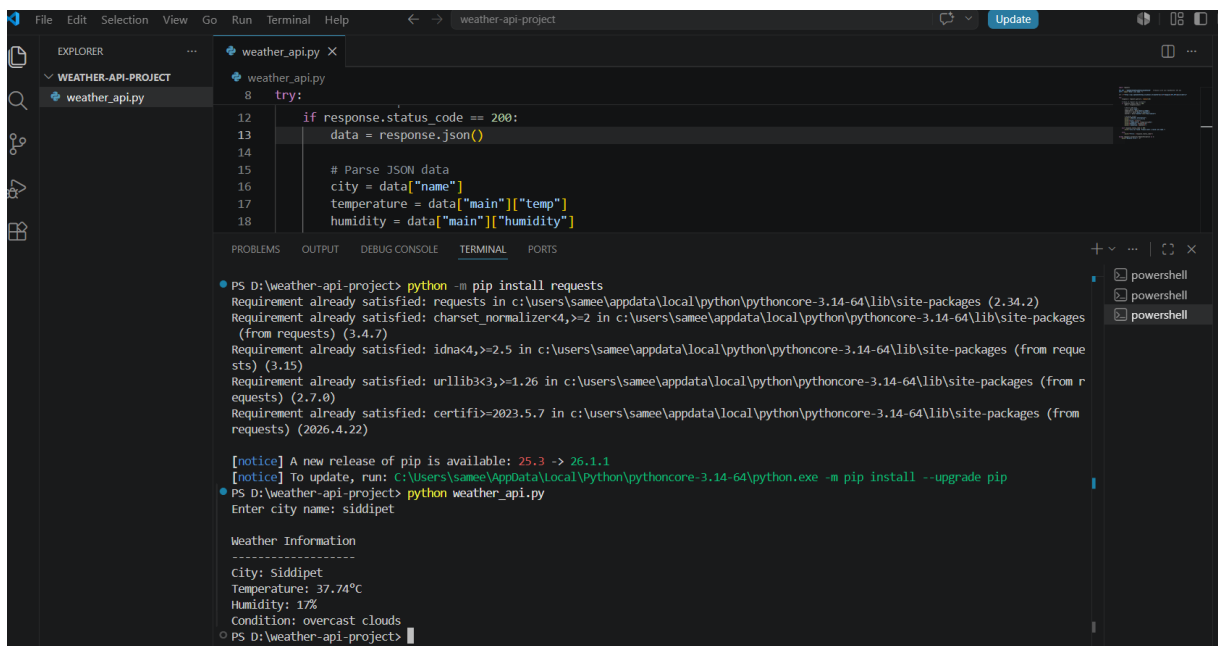


Screenshot 2: Python Code in VS Code



Screenshot 3: Successful Program Execution

Successful execution of the application showing weather details such as city name, temperature, humidity, and weather condition retrieved from the API.



Screenshot 4: JSON Data Processing

JSON data processing in Python. The API response is converted from JSON format into a Python dictionary using `response.json()`, and specific fields such as city name, temperature, humidity, and weather condition are extracted for display.

```

try:
    response = requests.get(url, timeout=10)

    # Check if request was successful
    if response.status_code == 200:
        data = response.json()

        # Parse JSON data
        city = data["name"]
        temperature = data["main"]["temp"]
        humidity = data["main"]["humidity"]
        weather = data["weather"][0]["description"]

        # Display output
        print("\nWeather Information")
        print("-----")
        print(f"City: {city}")
        print(f"Temperature: {temperature}°C")
        print(f"Humidity: {humidity}%")
        print(f"Condition: {weather}")

    elif response.status_code == 404:
        print("City not found. Please enter a valid city name.")

    else:
        print(f"Error: {response.status_code}")

except requests.exceptions.RequestException as e:
    print("Network error:", e)

```

Screenshot 5: Invalid City Error

Error handling demonstration when an invalid city name is entered. The program displays an appropriate error message instead of crashing.

```

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

● PS D:\weather-api-project> python weather_api.py
  Enter city name: Narayanraopet
  City not found. Please enter a valid city name.
○ PS D:\weather-api-project>

```

Screenshot 6: Network/API Error Handling

Exception handling implemented using try-except blocks to manage network issues and API connection failures gracefully.


```
● PS D:\weather-api-project> python weather_api.py
Enter city name: Hyderabad
Error: 401
○ PS D:\weather-api-project> █
```

Screenshot 7: Terminal Outputs

Terminal window displaying the execution process and weather information returned by the API for the user-provided city.

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
● PS D:\weather-api-project> python weather_api.py
Enter city name: siddipet

Weather Information
-----
City: Siddipet
Temperature: 37.74°C
Humidity: 17%
Condition: overcast clouds
● PS D:\weather-api-project> python weather_api.py
Enter city name: hyderabad

Weather Information
-----
City: Hyderabad
Temperature: 34.23°C
Humidity: 33%
Condition: haze
● PS D:\weather-api-project> python weather_api.py
Enter city name: karimnagar

Weather Information
-----
City: Karimnagar
Temperature: 40.15°C
Humidity: 16%
Condition: broken clouds
○ PS D:\weather-api-project> █
```

Conclusion

This project successfully demonstrated how Python communicates with external APIs using the Requests library. Weather data was retrieved from an external API, processed in JSON format, and displayed in a user-friendly manner. The project also implemented search functionality based on user input and handled errors such as invalid city names and network issues. Through this project, I gained practical experience in API integration, JSON parsing, data extraction, and exception handling in Python.